Guía de requisitos pendientes

Capacidades por completar en FastAPI

Incluye rutas existentes y trabajo requerido. Los endpoint propuestos son sugerencias de contrato, no rutas ya implementadas.

1. Recuperación de contraseña

Actual: POST /api/v1/auth/change-password requiere sesión y contraseña actual; no se encontró recuperación de cuenta.

- Agregar solicitud y confirmación de reset por correo/código de un solo uso, token almacenado con hash, caducidad e invalidación tras uso.
- Responder de forma genérica para evitar revelar cuentas; aplicar límites por cuenta/IP, auditoría y política de contraseñas. Incrementar TokenVersion tras el reset.

2. Creación de administradores

Actual: POST /api/v1/auth/register crea participantes; GET/PATCH /api/v1/admin/users permite listar/actualizar usuarios. No hay alta administrativa POST.

- Crear endpoint restringido a ADMIN para invitar/crear ADMIN o EVENT_MANAGER con correo validado y contraseña temporal o enlace de activación.
- Auditar actor, rol y activación; evitar alta pública ADMIN; definir alcance por competencia para EVENT_MANAGER. Conectar el modal visual del frontend a ese contrato.

3. Formularios dinámicos

Actual: PUT /api/v1/admin/events/{event_id}/registration-config acepta título, descripciones/instrucciones, métodos de pago, contacto, moneda y activación. Su esquema prohíbe propiedades extra y no acepta custom_fields.

- Persistir un esquema versionado de preguntas por evento: identificador, etiqueta, tipo, opciones, orden, requerido y validaciones.
- Incluirlo en GET/PUT administrativo y GET público; validar respuestas en servidor y guardar respuestas ligadas a la inscripción y a la versión del formulario.
- Hasta ampliar el esquema, el constructor frontend envía custom_fields y recibirá error 422; no se guardan las preguntas.

4. Cronómetro multicliente

Actual: GET/PUT /api/v1/competition/combate-actual y WebSocket /ws/combate-actual transmiten el combate. El cronómetro del frontend usa estado local, por lo que otros dispositivos no comparten duración, inicio ni pausa.

- Guardar configuración/estado del timer por combate en servidor: enabled, duración, started_at, pausa y tiempo restante.
- Definir operaciones autorizadas para iniciar/pausar/reiniciar y emitir cambios por WebSocket. Los clientes deben calcular el tiempo desde marcas de servidor para evitar deriva y recuperarse tras reconexión.

5. Combates, puntajes y clasificación

Actual: PUT /api/v1/competition/combate-actual actualiza estado y puntuación; GET /api/v1/events/{event_id}/leaderboard suma ScoreEntries de inscripciones aprobadas. Los estados FINISHED/CANCELLED no admiten transiciones.

- Si se necesita reabrir FINALIZADO EN_VIVO, habilitarlo como transición explícita, con permiso, motivo/auditoría y reglas para conservar o corregir puntajes.
- Publicar regla de desempate: ahora se usa rango denso, sin criterio deportivo adicional.

• Verificar que los snapshots/WebSocket y la tabla pública sigan actualizándose después de finalizar.

6. Criterios de aceptación

• Reset de contraseña de un solo uso invalida credenciales previas y queda auditado.
• Solo ADMIN crea cuentas administrativas y cada alta tiene trazabilidad.
• Preguntas y opciones sobreviven a guardar/leer; las respuestas se validan y persisten por inscripción.
• Dos navegadores conectados muestran el mismo cronómetro y retoman desde estado servidor.
• Los cambios de puntaje quedan auditados y la clasificación aplica reglas publicadas.

Acordar modelos, permisos, migraciones y contratos antes de implementar. Este documento no modifica el backend.